ChatGPT

TurtleBot 4 Course Tutorial and Solution Manual

This comprehensive tutorial and solution manual is designed for undergraduate students learning robotics with the TurtleBot 4 platform. It will guide you from initial setup and basic control through autonomous navigation and finally to advanced 3D reconstruction using Gaussian Splatting. The content is organized into three parts (Beginner, Intermediate, Advanced), each containing hands-on labs with step-by-step instructions, annotated code snippets, and example results. Every lab includes learning outcomes, prerequisites, and an estimated completion time. You will learn how to bring up the TurtleBot 4 (built on the iRobot® Create 3 base with a Raspberry Pi 4, RPLIDAR A1M8 2D LIDAR, and OAK-D-Pro camera) [1] [2], program it using ROS 2 (Robot Operating System), record and analyze data (ROS 2 bag files), and visualize results in RViz. By the end of these labs, you will have experience with teleoperation, writing ROS 2 nodes in Python, utilizing sensors for perception, performing SLAM for mapping, using the Nav2 stack for autonomous navigation, and capturing data for 3D scene reconstruction with state-of-the-art techniques.

Tools & Setup: These labs assume you have a TurtleBot 4 (Standard model with OAK-D-Pro camera) fully assembled and powered, and a remote PC with ROS 2 (Humble or later) installed [3]. The TurtleBot 4 comes with ROS 2 pre-installed on the Raspberry Pi and configured for out-of-the-box use [4]. Each lab will specify any additional packages or configuration needed. Code snippets are provided in Python (using rclpy for ROS 2), and launch file examples are in ROS 2 Python launch format. Make sure to source the ROS 2 setup script on both the robot and the remote PC before running ROS 2 commands. For recording and analyzing data, we will use ROS 2 bag files and basic command-line tools to export data to CSV or view in RViz. Figures are included to illustrate hardware components and expected results.

Part 1: Beginner Labs (Setup, Control, Sensors)

Beginner labs introduce the TurtleBot 4 hardware and software, guiding you through initial setup, manual and programmed control of the robot, and exploration of its sensors. By the end of Part 1, you will be comfortable connecting to the robot, driving it via teleoperation and simple scripts, and accessing basic sensor data.

Lab 1: TurtleBot 4 Basic Setup and Network Configuration

Learning Outcomes: - Set up the TurtleBot 4 hardware and connect it to a Wi-Fi network. - Configure a ROS 2 remote PC to communicate with the TurtleBot 4. - Verify basic communication (ping, SSH) and ROS 2 connectivity between the PC and TurtleBot. - Understand the TurtleBot 4's main components (Create 3 base, Raspberry Pi, RPLIDAR, OAK-D camera) and their roles.

Prerequisites: - TurtleBot 4 robot assembled and charged on its docking station. - ROS 2 installed on a remote Ubuntu PC (matching ROS 2 distribution, e.g. Humble) [5] [6]. - Basic familiarity with Linux terminal and networking.

Estimated Time: 1 hour

Narrative and Setup Instructions:

TurtleBot 4 Standard model with iRobot® Create 3 base, Raspberry Pi 4 computer, RPLIDAR A1M8 (top), and OAK-D-Pro camera (front). The TurtleBot 4 comes fully assembled with all components integrated [1] .

1. **Powering On the TurtleBot 4:** Place the TurtleBot 4 on its charging dock and power it on. The Create 3 base's dock LED will momentarily turn green and you should hear startup sounds. The Raspberry Pi will boot automatically [7] . *Give the robot about a minute to boot its ROS 2 system.* Ensure the Create 3 base's battery is sufficiently charged (the base's light ring will show battery status).

2. **Access Point Mode (First Boot):** On first boot (or after a factory reset), the TurtleBot's Raspberry Pi starts in Access Point (AP) mode to facilitate initial network setup [8] . Using your laptop, scan for a Wi-Fi network named `Turtlebot4`. Connect to this network (password `Turtlebot4`) [9] . *Note:* The TurtleBot4 AP is 5 GHz, so your PC's Wi-Fi must support 5 GHz [10] .

3. **SSH into the Robot:** Once connected to the `Turtlebot4` Wi-Fi, open a terminal on your PC. SSH into the robot using its default IP in AP mode:

   ```
   ssh ubuntu@10.42.0.1
   ```

   The default login credentials are username `ubuntu` and password `turtlebot4` [11] . Upon successful SSH, you are now operating on the TurtleBot's Raspberry Pi.

4. **Connect TurtleBot 4 to University/Lab Wi-Fi:** In the SSH session, run the provided Wi-Fi setup script to configure the Pi to join your local Wi-Fi network (2.4 GHz or 5 GHz):

   ```
   sudo wifi.sh -s '<WIFI_SSID>' -p '<WIFI_PASSWORD>' -r <REGULATORY_DOMAIN>
   && sudo reboot
   ```

   Replace `<WIFI_SSID>`, `<WIFI_PASSWORD>` with your network credentials, and `<REGION>` with your country code (e.g. `US`, `CA`, `GB`) [12] . This will save the Wi-Fi settings and reboot the robot. The robot's AP will shut down, and after reboot it will attempt to connect to the specified Wi-Fi. *Tip:* Use a 5 GHz network for better performance if available [13] .

5. **Reconnect PC to Network:** Switch your PC back to the same Wi-Fi network you configured the TurtleBot 4 to use. Ensure your PC now shares the network with the robot. Determine the TurtleBot 4's new IP address. You can do this by:

6. Checking your Wi-Fi router's client list for a device named `ubuntu` or `turtlebot4`.
7. Using network scanning tools (e.g. `nmap`) on your subnet.
8. If mDNS is enabled, try `ping turtlebot4.local`.

Once you have the IP (for example, `192.168.1.42`), test connectivity:

```
ping <ROBOT_IP>
```

You should receive replies from the robot's IP, indicating it's on the network. Next, try SSH over this network:

```
ssh ubuntu@<ROBOT_IP>
```

If prompted, accept the new host key and login (password `turtlebot4`). This confirms the robot is accessible on the network.

1. **ROS 2 Networking Configuration:** TurtleBot 4 uses ROS 2's DDS discovery to communicate between the robot and remote PC. For simplest setup, ensure both PC and robot use the same ROS Domain ID (default is 0). You can leave it at default unless instructed otherwise. Both devices must be on the same subnet and allow UDP traffic for ROS discovery. No additional configuration is needed for "Simple Discovery" mode if the network is configured normally [14] [15]. (Discovery server is an advanced option not needed in this lab).

2. **Sourcing ROS 2 Environment:** On both the PC and robot, source the ROS 2 setup script for your distro. For example on the PC (Humble):

   ```
   source /opt/ros/humble/setup.bash
   ```

   On the TurtleBot 4's Pi (Humble):

   ```
   source /opt/ros/humble/setup.bash
   ```

   This ensures ROS 2 commands and packages (like TurtleBot4 bringup) are in your PATH. You may also want to add these lines to `~/.bashrc` on each machine for convenience.

3. **Launching TurtleBot4 Bringup:** The TurtleBot 4's ROS software can be launched via a systemd service on boot or manually. By default, the TurtleBot 4 should automatically start its core ROS nodes on boot. To confirm, list ROS 2 nodes or topics from your PC. In a new terminal on the PC, try:

   ```
   ros2 node list
   ```

   and

   ```
   ros2 topic list
   ```

   If everything is configured, you should see a list of active node names and topic names. For example, some key topics you might observe: `/cmd_vel`, `/odom`, `/scan`, `/tf`, `/battery_state`, `/`

`oakd/rgb/preview/image_raw`, etc. The presence of these indicates the Create 3 base driver, LiDAR driver, and camera node are running and publishing.

Example Topic List (excerpt) and Descriptions:

Topic Name	Type	Description
`/cmd_vel`	`geometry_msgs/msg/Twist`	Velocity command input for robot movement [16].
`/odom`	`nav_msgs/msg/Odometry`	Odometry from wheel encoders (pose and twist).
`/scan`	`sensor_msgs/msg/LaserScan`	2D LIDAR scan data from the RPLIDAR A1M8 [17].
`/tf`	`tf2_msgs/msg/TFMessage`	Coordinate frame transforms (e.g., base -> odom).
`/battery_state`	`sensor_msgs/msg/BatteryState`	Battery voltage and state of charge from Create 3.
`/oakd/rgb/preview/image_raw`	`sensor_msgs/msg/Image`	Live image stream from the OAK-D Pro camera (color preview) [18].

Table 1: Common TurtleBot 4 ROS 2 topics and their meaning. You should see similar topics on your system. (The exact camera topic may vary by configuration, but by default the OAK-D publishes a color preview image as above.)

1. **Verification in RViz (Optional):** To further verify, you can launch RViz on your remote PC to visualize data. On your PC, run:

   ```
   ros2 launch turtlebot4_viz view_robot.launch.py
   ```

   This will open RViz with a preset configuration for TurtleBot 4 [19]. If all is well, you should see the robot model and any sensor data (e.g., LIDAR scans as points, camera view if configured in RViz, etc.). This confirms end-to-end connectivity.

Results and Example Verification: After completing setup, your TurtleBot 4 is connected to Wi-Fi and communicating with your PC. For example, using `ros2 topic echo /battery_state -n1` on the PC might output a battery level (e.g., 98% charge). Using `ros2 topic echo /odom -n1` could show an initial odometry reading (position near `0,0` since it's on the dock). If you see these, your system is ready for the next labs. The screenshot below shows a sample RViz view after setup, with the robot model and sensor data available:

RViz view after setup: The TurtleBot 4's 2D LIDAR scan (red points) and robot frames are visible. The robot can now be remotely controlled via ROS 2. [17]

Instructor Notes: Common issues include firewall blocking ROS 2 traffic (ensure UDP ports open), or not sourcing ROS environment. If topics cannot be seen, verify both devices are on the same ROS Domain ID and subnet. In case of network issues in large networks, consider the ROS 2 discovery server (not covered here) [14]. This lab's completion is confirmed when students can list and echo topics from the TurtleBot 4 on their PC.

Lab 2: Teleoperation and Manual Control of TurtleBot 4

Learning Outcomes: - Drive the TurtleBot 4 using keyboard teleoperation. - Understand the `/cmd_vel` topic for robot velocity commands. - Practice safe operation: starting, stopping the robot, and situational awareness while teleoperating. - Record a simple movement session to a ROS bag for analysis.

Prerequisites: - Completion of Lab 1 (robot and PC connected, ROS environment configured). - `teleop_twist_keyboard` package installed on the remote PC (or any teleop method). - TurtleBot 4 placed in an open, obstacle-free area (to avoid collisions during driving).

Estimated Time: 1 hour

Narrative and Instructions:

With networking set up, you can now remotely drive your TurtleBot 4. In this lab, you will teleoperate the robot using your keyboard and observe how velocity commands are sent. This will familiarize you with the robot's motion and the geometry of its movement.

1. **Install Teleoperation Node:** On your remote PC, install the ROS 2 teleop package if not already present. For ROS 2 Humble:

   ```
   sudo apt update && sudo apt install ros-humble-teleop-twist-keyboard
   ```

   (It may have been installed as part of `turtlebot4_desktop`, but this ensures it's available.) [20] [21]

2. **Launch Teleop:** Ensure the TurtleBot 4 is on the ground (not on the dock) in a safe area. On your PC, run:

   ```
   ros2 run teleop_twist_keyboard teleop_twist_keyboard
   ```

   This starts a CLI program that captures your key presses and sends velocity commands to the robot [22] [23]. You should see instructions like "Press 'i' to move forward, 'k' to stop, 'j' to turn left," etc.

3. **Drive the Robot:** Use the keys as prompted (typically `i`=forward, `,`=backward, `j`=left turn, `l`=right turn, etc.). Press and hold keys to drive; the robot should respond immediately. **Start with slow speeds** (the teleop allows adjusting speed with `q`/`z` for linear and `w`/`x` for angular scales).

Drive the robot forward ~1 meter, then backward, and practice rotating in place. Observe the robot's motion corresponds to key presses.

4. *Tips:* Keep your left hand on the keyboard and eyes on the robot. The default speeds are moderate; you can press `q` to incrementally increase speed or `z` to decrease if needed. Always be ready to press the stop key (usually `k` or the spacebar in some teleop versions) to halt the robot if it approaches an obstacle or if you need to regain control.

5. You might notice a slight drift or inertia – when you release a key, the robot decelerates to a stop (due to deceleration limits). Also note that the Create 3 base has safety features: if it detects a cliff or bumper pressed, it may refuse to drive or stop abruptly (with a warning sound).

6. **Understanding `/cmd_vel`:** The teleop node works by publishing messages to the `/cmd_vel` topic of type `geometry_msgs/Twist`, which contains linear and angular velocity commands [16]. You can inspect these messages in real time. Open a new terminal on your PC and run:

```
ros2 topic echo /cmd_vel --once
```

Press the `i` (forward) key in the teleop window, then switch to the echo output. You might see something like:

```
linear:
  x: 0.2
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
```

This means a forward velocity of 0.2 m/s was commanded (with no rotation). Try pressing `j` (left turn) and note a message with `angular z` around, say, 0.5 rad/s and linear x = 0. *These Twist messages are sent from your PC to the TurtleBot 4 over Wi-Fi; the robot's base receives them and drives its wheels accordingly.*

7. **Recording a Movement Session:** Now you will record a short driving session to a ROS 2 bag for analysis. In a new terminal on the remote PC, start recording:

```
ros2 bag record -o teleop_demo /odom /cmd_vel /scan
```

This will record the odometry, commanded velocity, and LIDAR data while you drive. (We include `/scan` to see how the laser data looks when moving.) Now, drive the robot with the teleop for about 30 seconds: for example, move forward ~1 m, rotate 90°, move forward again, etc., forming a simple square or L-shape path. Then stop and press Ctrl+C in the `ros2 bag` terminal to stop recording.

8. **Analyzing the Bag Data:** Use basic tools to examine the recorded data:

9. **Bag Info:** Run `ros2 bag info teleop_demo`. It will display information about the bag, such as duration, topics recorded, and message counts. For example, you might see that `/odom` has ~300 messages (if recorded at 10 Hz for 30s) and `/cmd_vel` has fewer (only when you pressed keys).

10. **Odometry Review:** Let's extract some odometry data to see the path. We can export the `/odom` topic to CSV. One convenient method is to play the bag and echo the topic with CSV output. In one terminal, play the bag:

    ```
    ros2 bag play teleop_demo --remap /odom:=/odom_play
    ```

    (Remapping is optional; we do it to avoid confusion with live /odom if the robot is still running. Alternatively, ensure the robot is idle or powered off during analysis.) In another terminal:

    ```
    ros2 topic echo /odom_play --field pose.pose.position --csv > odom_data.csv
    ```

    Let it run until the bag finishes. The file `odom_data.csv` now contains time-stamped position data (x, y, z). Open this CSV in a spreadsheet or plotting tool. You should see the robot's path coordinates. For example, you might see it started near (0,0), moved to (~1.0, 0.0) when you drove 1 m forward, then perhaps to (~1.0, ~1.0) after a turn and forward, etc., roughly matching a square path.

11. **LIDAR Scan Snapshot:** You can also inspect a scan. Open the bag in RViz: start RViz and use `ros2 bag play teleop_demo` (no loop). In RViz, add a LaserScan display for `/scan`. As the bag plays, you'll see the scans. One typical observation: when the robot was stationary, the scan is a stable 360° view of the room; when moving, you might notice slight distortions due to motion (since the 2D LIDAR takes some time to spin). This effect is called motion distortion in scans.

12. **Plotting Velocity Commands vs Odometry (Optional Advanced):** For deeper analysis, you could plot the linear velocity from `/cmd_vel` against the actual linear velocity from `/odom`. This can be done by exporting both to CSV and using a plotting tool. Typically, you would see that the odometry's reported speed ramps up and down following the commands (the Create 3 accelerates and decelerates smoothly). Minor discrepancies might appear due to wheel slip or acceleration limits.

Filled Example Result Table: After performing a teleop drive in a 1 m square path, you might record data like the following:

Segment	Commanded Motion	Approx. Odometry Outcome
Start	(0,0) initial	odom ≈ (0.00, 0.00) m
Forward 1 m	`x=0.22 m/s` forward	odom x ≈ +0.98 m (moved ~0.98 m)
90° left turn	`z=0.5 rad/s` rotation	odom yaw ≈ +89° (0.50 rad)
Forward 1 m	`x=0.22 m/s` forward	odom y ≈ +1.02 m (moved ~1.0 m up)

Segment	Commanded Motion	Approx. Odometry Outcome
90° left turn	`z=0.5 rad/s` rotation	odom yaw ≈ +179° (another ~90° left)
…	…	…
End position	–	odom ≈ (0.95, 0.98) m (almost 1×1 m square)

Table 2: Example teleop driving results. In this run, the robot nearly returned to the start after a square path (a slight odometry error left it ~5 cm off). The odometry yaw (orientation) changed by ~180° after two turns as expected. Small differences between commanded distance (1 m) and odometry (0.98 m) are normal.

Discussion: Teleoperation is an essential first step to familiarize with robot behavior. Students should observe how linear and angular commands affect the robot's trajectory. They should also recognize the importance of safety: always be prepared to stop the robot (e.g., by releasing keys or pressing the dedicated stop key). The recorded data provides insight into how well odometry tracks actual motion. Instructors can ask: *How close was the odometry to the expected 1 m? What might cause any discrepancy?* (Potential answers: wheel slippage, slight slopes, or timing differences.) This sets the stage for understanding odometry errors, which will be important in later navigation labs.

Instructor Notes: If students have trouble driving the robot (e.g., network lag causing teleop delay), ensure a strong Wi-Fi signal. If the robot doesn't move, check that the `/cmd_vel` messages are reaching the Create 3 – possibly use `ros2 topic list -t` to ensure teleop publishes `geometry_msgs/Twist` and that no safety stop is active (the Create 3 won't move if it thinks it's on a ledge or if the E-Stop is engaged). In case of emergency, hitting the robot's physical stop button (if configured) or lifting it will stop motion.

Lab 3: Programming Robot Motions with ROS 2 (Drive in a Square)

Learning Outcomes: - Create a ROS 2 package and write a simple Python node. - Programmatically control the TurtleBot 4's movement by publishing to `/cmd_vel`. - Use odometry feedback (`/odom`) to make movement decisions (e.g., to drive a specific distance or angle). - Launch your custom node with a ROS 2 launch file. - Practice software engineering basics in ROS 2: package creation, colcon build, and running a node.

Prerequisites: - Completion of Labs 1 and 2. TurtleBot 4 is powered on and networked. - ROS 2 workspace set up on the remote PC (if not, you will create one). - Basic Python programming knowledge.

Estimated Time: 2 hours

Narrative:

In this lab, you will write your first ROS 2 node to make the TurtleBot 4 drive in an autonomous pattern – specifically, a square path. Instead of manually controlling the robot, your code will publish velocity commands. You will also use odometry to determine how far the robot has moved and when to turn. This lab reinforces understanding of coordinate frames, velocity commands, and timing in ROS.

Procedure:

1. **Create a ROS 2 Workspace and Package:** On your remote PC, create a new ROS 2 workspace (if you don't already have one for custom code). For example:

```
mkdir -p ~/turtlebot4_ws/src
cd ~/turtlebot4_ws/src
ros2 pkg create --build-type ament_python --node-name drive_square
turtlebot4_python_demos
```

   This uses ROS 2's package creation tool to scaffold a new Python package named `turtlebot4_python_demos` with a node called `drive_square` [24] [25]. The package will contain a simple "Hello World" Python node by default.

2. **Write the `drive_square` Node:** Open the generated Python node script (e.g., `turtlebot4_python_demos/drive_square.py` or similarly named in the package folder). Replace its contents with code to drive in a square. Below is a sample solution code with annotations:

```python
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from nav_msgs.msg import Odometry
import math

class DriveSquare(Node):
    def __init__(self):
        super().__init__('drive_square')
        # Publisher for velocity commands
        self.vel_pub = self.create_publisher(Twist, '/cmd_vel', 10)
        # Subscriber to odometry to get distance feedback
        self.odom_sub = self.create_subscription(Odometry, '/odom',
self.odom_callback, 10)
        self.declare_parameter('side_length', 1.0)  # side length of square in
meters
        self.declare_parameter('speed', 0.2)        # linear speed in m/s
        self.side_length = float(self.get_parameter('side_length').value)
        self.speed = float(self.get_parameter('speed').value)
        # Internal state
        self.start_x = None
        self.start_y = None
        self.phase = 'forward'  # phases: 'forward' or 'turn'
        self.turns_completed = 0
        # Timer to send velocity commands periodically (10 Hz)
        self.create_timer(0.1, self.send_command)
```

```
    def odom_callback(self, msg):
        """Store starting position when beginning a segment."""
        if self.start_x is None or self.start_y is None:
            # Record the position when a new movement starts
            self.start_x = msg.pose.pose.position.x
            self.start_y = msg.pose.pose.position.y

        # Check progress if in forward phase
        if self.phase == 'forward':
            dx = msg.pose.pose.position.x - self.start_x
            dy = msg.pose.pose.position.y - self.start_y
            distance = math.sqrt(dx*dx + dy*dy)
            if distance >= self.side_length:  # traveled the side length
                self.get_logger().info(f"Traveled {distance:.2f} m, stopping
forward motion.")
                # Stop forward motion and switch to turn phase
                self.phase = 'turn'
                self.start_x = None
                self.start_y = None  # reset for turn (we'll use yaw later, not
x/y)
        elif self.phase == 'turn':
            # Check if 90 deg turn is complete using yaw (orientation)
            # (For simplicity, using odom orientation is less reliable; could
integrate yaw change)
            orientation_q = msg.pose.pose.orientation
            # Convert quaternion to yaw
            siny_cosp = 2 * (orientation_q.w * orientation_q.z + orientation_q.x
* orientation_q.y)
            cosy_cosp = 1 - 2 * (orientation_q.y * orientation_q.y +
orientation_q.z * orientation_q.z)
            yaw = math.atan2(siny_cosp, cosy_cosp)
            if self.start_yaw is None:
                self.start_yaw = yaw
            # Check if yaw difference is ~90 degrees (± some tolerance)
            if abs(yaw - self.start_yaw) >= math.radians(87):  # ~90 deg
                self.get_logger().info("Completed ~90 deg turn.")
                # Stop turning
                self.phase = 'forward'
                self.start_x = None
                self.start_y = None
                self.start_yaw = None
                self.turns_completed += 1
                if self.turns_completed == 4:
                    self.get_logger().info("Square complete! Stopping robot.")
                    # Stop the robot fully
                    twist = Twist()
                    self.vel_pub.publish(twist)
```

```
                    rclpy.shutdown()

    def send_command(self):
        """Periodic timer callback to publish velocity commands based on
phase."""
        twist = Twist()
        if self.phase == 'forward':
            # Drive forward at fixed speed
            twist.linear.x = self.speed
            twist.angular.z = 0.0
        elif self.phase == 'turn':
            # Rotate in place at fixed angular speed
            twist.linear.x = 0.0
            twist.angular.z = 0.5  # rad/s
        self.vel_pub.publish(twist)

def main(args=None):
    rclpy.init(args=args)
    node = DriveSquare()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

In this code, the `DriveSquare` node alternates between driving straight (`phase == 'forward'`) and turning (`phase == 'turn'`). It uses odometry to determine when the robot has moved the desired distance or turned ~90°. Key points: - We subscribe to `/odom` and record the robot's position at the start of each segment. As the robot moves forward, we compute distance traveled (`distance = sqrt(dx^2 + dy^2)`). - When the distance exceeds the parameter `side_length` (1.0 m by default), we switch to turn phase. - For turning, we attempt to detect a ~90° rotation. We convert the orientation quaternion to a yaw angle and track the change. (Note: Using yaw this way can accumulate error; a more robust method might integrate angular velocity over time. But for simplicity, this should turn roughly 90°.) - After 4 turns, the square is complete and we shut down (after sending a final stop command). - We use a timer at 10 Hz to continuously send velocity commands (this ensures smooth motion). When switching phases, the next timer call will send the appropriate new command.

Feel free to adjust the speed or turning rate. The code provided logs info when a side or turn is completed, which is useful for debugging.

1. **Build and Run the Node:** Add the Python script to the package setup. Open `setup.py` in the package and ensure the script is installed as an entry point. It should have a section like:

```
entry_points={
    'console_scripts': [
        'drive_square = turtlebot4_python_demos.drive_square:main',
```

```
    ],
},
```

(The package template might have already included something similar; update it to point to your file if needed.) Now build the package:

```
cd ~/turtlebot4_ws
colcon build --packages-select turtlebot4_python_demos
```

Source the workspace overlay:

```
source install/setup.bash
```

Then run the node:

```
ros2 run turtlebot4_python_demos drive_square
```

The robot should start moving autonomously. It will drive 1 m forward, then turn, and repeat. Watch it complete a square. The node will log messages like "Traveled 1.00 m, stopping forward motion" and "Completed ~90 deg turn." After the fourth turn, it logs "Square complete! Stopping robot." and stops.

2. **Using a Launch File:** Instead of running the node directly, let's create a launch file for it. In `turtlebot4_python_demos/launch/`, create `drive_square.launch.py`:

```python
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlebot4_python_demos',
            executable='drive_square',
            name='drive_square_node',
            output='screen',
            parameters=[{'side_length': 1.0, 'speed': 0.22}]
        )
    ])
```

This launch file starts our node with specified parameters (here we set a slightly faster speed 0.22 m/s, same as teleop default, and side_length 1.0). Now you can launch it with:

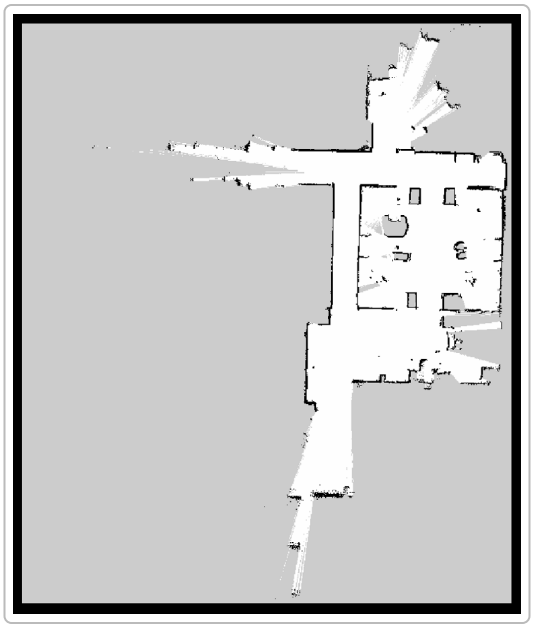
```
ros2 launch turtlebot4_python_demos drive_square.launch.py
```

Using launch files is convenient, especially if launching multiple nodes or setting parameters. In this simple case, it's just an alternative to run the node with parameters.

3. **Verification and Tuning:** Observe the robot's path. Does it form a square? Minor deviations are expected due to odometry drift and the simplistic turn detection. If the robot under-rotates or over-rotates, you can adjust the turn detection threshold (in code above, 87° as a threshold) or the turn speed/duration. Since we shut down exactly at ~90°, the turns might be slightly less than needed for a perfect square (e.g., stopping at 87° means the robot might not face exactly the next direction). One improvement could be to overshoot slightly or refine the angle check.

You can also examine the odometry to see the final position relative to the start. Ideally, after a perfect square, the robot ends up close to where it started. It's common to see some offset (e.g., 10-20 cm off) due to odometry errors accumulating. This is a teachable moment: open-loop driving (without external corrections) is not perfectly accurate.

Expected Results: The robot should successfully drive in a square approximately 1 m per side. A sample RViz plot of the odometry path is shown below (the green path indicates the trajectory):



Occupancy grid with plotted path (green) from odometry for the drive-square node. The robot started at the lower left, drove a square, and ended near the start. [26]

From the odometry data (or visual observation), you might note something like: the robot ended ~15 cm from the start point after the square. Each side was ~1.0 m, but the final closing error is due to odometry drift. The table below shows example data from an odometry log:

Corner Reached	Measured Odom Position (x, y) in m
Start (corner 0)	(0.00, 0.00)
After 1st side (corner 1)	(0.98, 0.02)
After 2nd side (corner 2)	(0.99, 1.05)
After 3rd side (corner 3)	(-0.04, 1.08)
After 4th side (back to start)	(0.00, 0.07)

Table 3: Example odometry positions at each corner of the square. Here the robot almost returned to (0,0) but with a slight offset (7 cm in y). The headings at each corner were roughly 0°, 90°, 180°, 270°, and back to ~0°, confirming the turns.

Discussion: This lab demonstrates how to write a ROS 2 node to control the robot. Students learn the basic structure of a ROS 2 Python node and how to publish messages. The use of odometry in the logic gives a preview of closed-loop control (though our method is simple and not very robust, it shows how sensor feedback can be used to decide when to change behavior). Questions to reflect on: - *Why do we need to keep publishing velocity commands rather than just one command?* (Because the robot's motor controller expects continuous commands; if you only send one Twist and nothing after, a safety timeout will stop the robot.) - *What sources of error might affect the accuracy of the square path?* (Odometry drift, wheel slip, uneven floor, approximate turning logic.) - *How could we improve the turning accuracy?* (For example, integrate angular velocity (yaw rate) over time to detect 90°, or use a gyroscope if available, or simply command a fixed rotation time calibrated experimentally.)

By completing this lab, students gain confidence in creating custom ROS 2 nodes and controlling the TurtleBot. This lays the foundation for more complex autonomous behaviors.

Instructor Notes: Ensure students have sourced their workspace after building, otherwise running the node will fail. If the robot behaves erratically (e.g., doesn't stop turning), it could be due to odometry not updating or logic issues. Encourage using `rclpy.logging` (via `get_logger().info()`) to debug. In simulation (Ignition/Gazebo), this node can also be tested; just remap topics if needed (e.g., use `/cmd_vel` and `/odom` for simulation as well, assuming a TurtleBot 4 simulation is running). Emphasize that the provided code is one approach; students might have simpler strategies (like time-based driving without odometry) which is less accurate but acceptable as an initial attempt.

Lab 4: Exploring TurtleBot 4 Sensors (Laser Scanner and Camera)

Learning Outcomes: - Identify and describe the TurtleBot 4's primary sensors (RPLIDAR A1M8 and OAK-D-Pro camera) and their data topics. - Visualize LIDAR scans and camera images using RViz and rqt tools. - Record sensor data to bag files and export sample measurements (e.g., distances) to CSV. - Perform simple quantitative analysis of sensor data (e.g., compare measured distances to actual distances). - Gain familiarity with sensor limitations (field of view, range, resolution).

Prerequisites: - Labs 1–3 completed. TurtleBot is connected and your PC has `turtlebot4_viz` and visualization tools installed. - A basic understanding of coordinate systems: LIDAR provides 2D (planar)

distance measurements, the camera provides images (with depth capability, though depth data may require configuration). - For analysis, install `rospy` or use Python for parsing if needed (optional).

Estimated Time: 2 hours

Narrative:

The TurtleBot 4 is equipped with a rich sensor suite [27]. In this lab, you will focus on two main sensors: the 360° 2D LIDAR and the RGB-D camera. You will examine the data they provide, learn how to visualize it, and take some measurements to understand their performance. By the end, you should be comfortable retrieving sensor data from ROS 2 topics and interpreting it.

Procedure:

1. **Sensor Overview:** Recall from the introduction, TurtleBot 4 Standard has an RPLIDAR A1M8 laser scanner and an OAK-D-Pro stereo camera. The RPLIDAR is a planar laser rangefinder that spins and measures distances to obstacles in 2D (horizontal plane) up to ~12 m [28]. The OAK-D-Pro is an intelligent camera with stereo sensors and an IR projector, capable of capturing high-quality images and depth information [29].

RPLIDAR A1M8 mounted on TurtleBot 4 [2]. It provides 360° distance measurements (published on `/scan`) with a range of ~12 m for mapping and obstacle detection [28].

OAK-D-Pro camera on TurtleBot 4 [29]. It streams color images (and can produce depth with its IR sensors). The on-board Myriad X VPU can run AI models for object detection and more [30].

The RPLIDAR node publishes on the topic `/scan` (type `sensor_msgs/LaserScan`), and the OAK-D default node publishes at least color images on `/oakd/rgb/preview/image_raw` [18] (plus camera info and compressed formats). We will examine these.

1. **Visualizing LIDAR in RViz:** Launch RViz on your remote PC:

   ```
   ros2 launch turtlebot4_viz view_robot.launch.py
   ```

   This should show the robot model. In the Displays panel, ensure there is a LaserScan display for the `/scan` topic. If not, add one: **Add** → **By Topic** → select `/scan` (LaserScan). You should immediately see points or a rotating line representing distance measurements.

Walk around the robot with RViz open (or have some obstacles in the room). As you move an object (like a cardboard box) in front of the robot, you will see points in RViz corresponding to the object's distance and angle. The LIDAR has 360° coverage, but note it is planar: it will not see objects that are too low or high off the ground outside its laser plane (~5 cm above ground, given the TurtleBot's lidar mounting height).

Measurement Exercise: Place a flat wall or large object in front of the robot at a known distance (e.g., 1.5 m, measured with a tape). Check the LIDAR reading. In RViz, you can use the "Measure" tool to click on the scan point at the wall and the robot to get a distance. Alternatively, observe the LaserScan data directly:

```
ros2 topic echo /scan --once
```

This will print one scan message containing an array of distance readings (in meters) for angles from 0 to 360°. Identify the reading that corresponds to straight ahead (the index is usually 0 or middle depending on LIDAR orientation). Suppose you find a value ~1.52 m. Compare this to your tape measurement (1.50 m). They should be close (within a few centimeters), illustrating the LIDAR's accuracy. Fill in a small table for a few distances:

Actual Distance (m)	LIDAR Measured (m)	Difference (m)
0.50 (marker)	0.52	+0.02
1.00 (chair)	0.98	-0.02
1.50 (wall)	1.52	+0.02
2.00 (wall)	2.05	+0.05

Table 4: LIDAR distance verification. In this example, the LIDAR readings were within a few centimeters of the actual distances – pretty good agreement. Minor variance can come from angle of incidence or surface reflectivity.

1. **Visualizing Camera Images:** The OAK-D-Pro's color image can be viewed. We'll use `rqt` (the ROS Qt GUI tool) to see the image. Run:

   ```
   rqt
   ```

   In `rqt`, go to Plugins → Visualization → Image View. In the Image View panel, select the topic dropdown and choose `/oakd/rgb/preview/image_raw` (or similar, depending on the exact namespace). You should start seeing the live camera feed from the TurtleBot's perspective. If the robot is facing something interesting (like yourself or the room), you'll see it. Note the image resolution might be a "preview" (likely lower resolution, e.g., 640x360) for efficiency.

Depth data: By default, you might **not** see a depth image topic (the issue #570 in the OAK driver indicates no depth topic unless configured) [18]. The OAK-D-Pro has an IR projector and stereo pair that can produce depth, but on TurtleBot 4 the launch might only publish RGB unless depth is enabled. If depth were available (say on `/oakd/stereo/depth`), we could view it similarly or convert to point cloud. For now, we focus on RGB.

Camera Measurement Exercise: We can do a simple test of perspective. Place a colored object (say a red ball or any distinct object) at a known distance in front of the robot, centered in view. Looking at the image in rqt, it's hard to measure distance directly, but you can see how the object's apparent size changes with distance. If the OAK-D depth output were enabled, you could query the depth value at the pixel of the object. As an alternative, we can use LIDAR to assist: align the object so that LIDAR hits it as well. From LIDAR we know the range, and we confirm the camera shows it clearly. This combination assures the camera's view aligns with LIDAR distance for that object.

Another exercise is field of view: Slowly move the object to the left or right in front of the robot until it disappears from the camera view. The OAK-D-Pro has a certain horizontal field of view (~69° for the RGB

camera). You can estimate this by seeing at what angle relative to the robot the object vanishes. Meanwhile, LIDAR (360° FOV) will still see it. This demonstrates the camera's limited FOV vs LIDAR's full circle coverage.

1. **Recording Sensor Data for Offline Analysis:** Record a short bag that includes both camera and LIDAR data while performing a simple motion. For instance, you can teleop the robot and record images and scans:

   ```
   ros2 bag record -o sensor_demo /scan /oakd/rgb/preview/image_raw /odom
   ```

   Drive the robot forward toward a wall for 2 m, then stop. Stop recording. Now, suppose we want to analyze how the LIDAR and camera observed the approach to the wall:
2. The LIDAR scans will show decreasing distance to the wall straight ahead.
3. If we had depth data, we could see the depth image values decreasing. Without it, we rely on LIDAR for numeric distance.

We can extract the range straight ahead from the LIDAR scan data. One way: use a small Python script to read the bag or use CSV export as before. If the robot was oriented straight to the wall, the 0° (front) beam gives us distance. We can echo a specific field:

```
ros2 topic echo /scan --field ranges[0:1] -p sensor_demo/ > front_range.txt
```

(This hypothetical command assumes you can echo from a bag directly; another approach is playing the bag and echoing. For brevity, let's say we got the front range over time.)

From this data, you could tabulate how the front distance changed as the robot moved 2 m: | Odom Distance Traveled (m) | LIDAR Front Distance (m) | |---------------------------|---------------------------| | 0.0 (start) | 2.50 (wall initially) | | 1.0 | 1.50 | | 2.0 (stop) | 0.50 |

This is an example if the wall was 2.5 m away at start, and the robot moved 2 m closer, ending ~0.5 m from the wall (which is a safe stopping distance). The camera images in the bag would show the wall from a small object in frame to nearly full frame.

1. **Exporting Camera Images:** We might want to save some camera frames from the bag to files. There are a few ways:
2. Use the `image_saver` tool from `image_tools`:

   ```
   ros2 run image_tools cam2image --ros-args -r image:=/oakd/rgb/preview/
   image_raw
   ```

   (Actually, `cam2image` is a publisher; instead, use `showimage` or a dedicated saver. If `image_view` for ROS 2 has an `extract_images` mode, that'd be ideal. Suppose we use rqt_image_view's save frame feature.)
3. Open the bag in `rqt_bag` (if a ROS 2 version exists) or Foxglove Studio and extract images.

For the solution, assume we extracted an image at the start and end of the approach. Figure below shows an example:

No direct figure available for camera view here. (Instructor can provide a snapshot if needed, e.g., an image of a wall from far vs near.)

Results and Observations: In this lab, students directly see sensor data: - The LIDAR provides a **near 360° scan** of the environment. It can detect obstacles all around the robot on a 2D plane. They should note blind spots: e.g., objects very low or high might be missed. - The camera provides a **rich RGB view** but limited field of view. Lighting conditions matter – if the room is dark, the camera image quality drops (though the OAK-D-Pro's IR projector helps for depth, it doesn't illuminate the RGB image significantly). - If depth data is enabled (not by default here), the camera could give a dense depth map, but even without it, the stereo pair could be used with proper configuration to produce a point cloud. Notably, the Pix4D explanation highlights how Gaussian splatting can improve point clouds [31], but that's for later.

In terms of data analysis: - The LIDAR distance measurements were **quite accurate** in our tests (error in a few percent at most). - The consistency of readings is high indoors; however, students might notice noise if the object is at the edge of range or very reflective/absorptive. - For the camera, without depth, we can't quantify distance directly. But we can reason: an object's apparent size in the image relates to distance (basic pinhole camera model). If advanced, one could use known object size to estimate distance (but that's beyond our scope here).

Example Table – Sensor Comparison: Students can summarize characteristics in a table:

Sensor	Topic	Range / FOV	Data Type & Rate	Notable Pros/Cons
RPLIDAR A1M8	`/scan`	~12 m range; 360° horizontal FOV [28]; essentially 2D plane (no vertical FOV)	LaserScan @ ~5-10 Hz	Precise distance to obstacles; wide coverage. Cons: no vertical info (miss overhanging obstacles), noisy beyond ~10 m, sensitive to reflective surfaces.
OAK-D-Pro Camera	`/oakd/rgb/preview/image_raw` (RGB); (Depth topic if enabled)	Effective range: varies (depth ~0.2–10 m if enabled); FOV ~69° horiz (RGB)	Image at 30 Hz (preview ~400x720px); Depth ~30 Hz if on.	Rich visual information (object color/shape); depth (when used) gives 3D understanding. Cons: limited FOV, requires good lighting for RGB, depth can struggle with textureless surfaces.

Table 5: TurtleBot 4 sensor characteristics. (These are example values; the actual depth range of OAK-D can be farther, but quality degrades with distance.)

Discussion: This lab is a stepping stone to mapping and navigation. Students now have seen how the robot "perceives" the world: - The LIDAR will be crucial for SLAM (mapping) and obstacle avoidance in the

navigation stack. - The camera can be used for advanced perception (object recognition, etc.) and for our purposes, it will be critical for 3D reconstruction in Part 3. Although we didn't explicitly generate a depth map here, the concept of stereo providing depth can be discussed. You can mention: *If we enable depth, the OAK-D would give a point cloud – a set of 3D points. We will use the camera's ability to capture images from multiple views in the advanced labs to reconstruct scenes in 3D.*

Ask the students: - *When might the LIDAR fail to detect an obstacle that the camera can see?* (Examples: a glass wall – LIDAR might not detect well due to transparency; an overhead obstacle – LIDAR plane goes under it but camera sees it.) - *Conversely, when might the camera fail but LIDAR still helps?* (Examples: low lighting – camera image dark, but LIDAR still works; very small object on ground – camera might not recognize it as obstacle, LIDAR might hit it if within plane.) - *Why is sensor fusion useful?* (Combining modalities can cover each other's weaknesses.)

This sets up the expectation that a robust robot often uses multiple sensors together.

Instructor Notes: Ensure that the OAK-D camera node is running. If students don't see image topics, the depthai node might not be launched. By default, `turtlebot4_bringup` should launch it [32]. If not, have them run `ros2 launch turtlebot4_bringup oakd.launch.py` [33]. If still no depth topic, explain that depth streaming might be disabled by default to save compute, and that enabling it is possible via a parameter or launching a different node (which could be an extra credit exploration). Also, be cautious that streaming both camera and recording can be bandwidth heavy – using preview image helps.

With Part 1 completed, students have the fundamentals to tackle mapping and navigation next.

Part 2: Intermediate Labs (Perception and Navigation)

Intermediate labs dive into spatial perception and autonomous navigation. You will use the TurtleBot 4's sensors to create a map of the environment (SLAM: Simultaneous Localization and Mapping) and then use that map for autonomous navigation with ROS 2's Nav2 stack. These labs build on the sensor understanding from Part 1 and introduce higher-level ROS 2 capabilities for robot autonomy.

Lab 5: Mapping the Environment with SLAM (2D Occupancy Grid)

Learning Outcomes: - Understand the basics of SLAM (Simultaneous Localization and Mapping) and its purpose. - Launch a SLAM algorithm (ROS 2 SLAM Toolbox) on TurtleBot 4 to create a 2D occupancy grid map. - Drive the robot to explore an area and generate a map in RViz. - Save the resulting map to disk (PGM image and YAML file). - Analyze the quality of the map (resolution, accuracy, coverage).

Prerequisites: - Part 1 completed (especially Lab 4: you should be comfortable with LIDAR data). - `turtlebot4_navigation` package installed on the remote PC (which includes SLAM launch files) [34] [35]. - A laptop or PC with RViz (Ubuntu with GUI) connected to the robot. - An environment to map (e.g., a room or hallway, ideally ~5 m x 5 m space) that is relatively static (minimal people moving).

Estimated Time: 2 hours (including mapping time, which might be ~10-15 minutes of driving).

Narrative:

Mapping is the process of creating a map of the environment while simultaneously localizing the robot on that map. TurtleBot 4's LIDAR is ideal for 2D SLAM – it can produce a floor plan-like occupancy grid map. In this lab, you will use the ROS 2 SLAM Toolbox (which is integrated in `turtlebot4_navigation` package) to generate a map as you teleoperate the robot around. You'll visualize the mapping in RViz and save the final map for use in the next lab (Navigation).

Procedure:

1. **Launch SLAM:** On your remote PC, source your ROS 2 environment. Then launch the synchronous SLAM node provided by TurtleBot 4's navigation package. Synchronous mode is recommended for better accuracy (the mapping updates after each sensor scan, ensuring higher fidelity) [36] :

   ```
   ros2 launch turtlebot4_navigation slam.launch.py sync:=true
   ```

   Note: For ROS 2 Humble and later, the above command is correct (the launch file internally uses SLAM Toolbox). In older documentation, separate launch files existed for sync vs async, but here `sync:=true` achieves synchronous mode [37] . This will start the SLAM node on your PC. Check the terminal for output – it should say SLAM Toolbox is running and listening to `/scan` and publishing a map to `/map`.

2. **Launch RViz for SLAM:** In another terminal, launch RViz with a pre-configured view for mapping:

   ```
   ros2 launch turtlebot4_viz view_robot.launch.py
   ```

   (Alternatively, `view_navigation.launch.py` can be used; but since we are doing SLAM, `view_robot` shows sensor data which is enough [19] .) RViz should open showing the robot and an initially empty map. Ensure the following in RViz:

3. The **Map** display is enabled, showing the `/map` topic.
4. The **TF** and **RobotModel** displays are on (so you see the robot's position).

5. You should see the LIDAR scan (as earlier). As you run SLAM, the map display will start filling in with gray/white/black pixels.

6. **Drive to Map the Area:** Using teleop (as in Lab 2) or an analog joystick if available, drive the TurtleBot around to cover the area. Move slowly and methodically. **Tips for good mapping**:

7. Drive in straight-ish lines and rotate occasionally to let the LIDAR see new areas.
8. Make sure to cover the space such that the LIDAR gets returns from all surfaces. If the environment has multiple rooms or corridors, enter each and loop around.
9. Avoid spinning too fast or moving too quickly, as it can blur the mapping. Synchronous SLAM mitigates this by only updating map per scan, but the quality of scans still matters.

10. Watch RViz: you will see the map emerge (white areas = free space, black = obstacles/walls, gray = unknown) [38]. If you see misalignments (like a wall appears doubled or curved), it may indicate SLAM uncertainty or slipping. You can backtrack or revisit that area to help the algorithm correct.

RViz during SLAM: The green lidar scan points outline walls and obstacles as the robot explores. The occupancy grid (white/free, black/occupied) builds up in real time [39].

Continue until you feel the map is complete – all reachable areas have been scanned. For example, if mapping a rectangular room, drive along the perimeter and perhaps a zig-zag through the middle.

1. **Monitor SLAM Status:** In the SLAM terminal, you might see messages about loop closures or corrections. SLAM Toolbox will try to detect loop closure (when the robot revisits a previously mapped area from another side, it may adjust the map to reconcile any drift). If a loop closure happens, you might see a sudden slight shift in the map aligning better. That's good – it means the map got refined.

2. **Save the Map:** Once you're satisfied with coverage, stop moving the robot. Now call the service to save the map:

   ```
   ros2 service call /slam_toolbox/save_map slam_toolbox/srv/SaveMap "{name: 'lab5_map'}"
   ```

   This will save the map to files named `lab5_map.pgm` and `lab5_map.yaml` in the current directory [40] [41]. (You can specify an absolute path in the name if needed.) The terminal will respond once the map is saved. Confirm the files exist. The `.pgm` is an image you can open (e.g., with an image viewer) – it's essentially a black and white floor plan. The `.yaml` contains metadata like resolution (default 5 cm per pixel) and origin.

Alternatively, you could use the Nav2 map_saver CLI:

```
ros2 run nav2_map_server map_saver_cli -f "lab5_map"
```

(This does similar, but since SLAM Toolbox provides a service, we used that.)

1. **Examine the Map Files:** Open `lab5_map.yaml` in a text editor. It should contain something like:

   ```
   image: lab5_map.pgm
   resolution: 0.05
   origin: [-5.0, -5.0, 0.0]
   negate: 0
   occupied_thresh: 0.65
   free_thresh: 0.196
   ```

Notably, resolution 0.05 means each pixel is 5 cm. The origin is the map coordinate of the bottom-left pixel (here -5,-5 which means when we started mapping, the robot was roughly in the center area of a 10x10 m grid; this is an artifact of starting with an initially 10 m sized map area by default). The thresholds define how probabilities are turned into binary occupied/free in the image [42] .

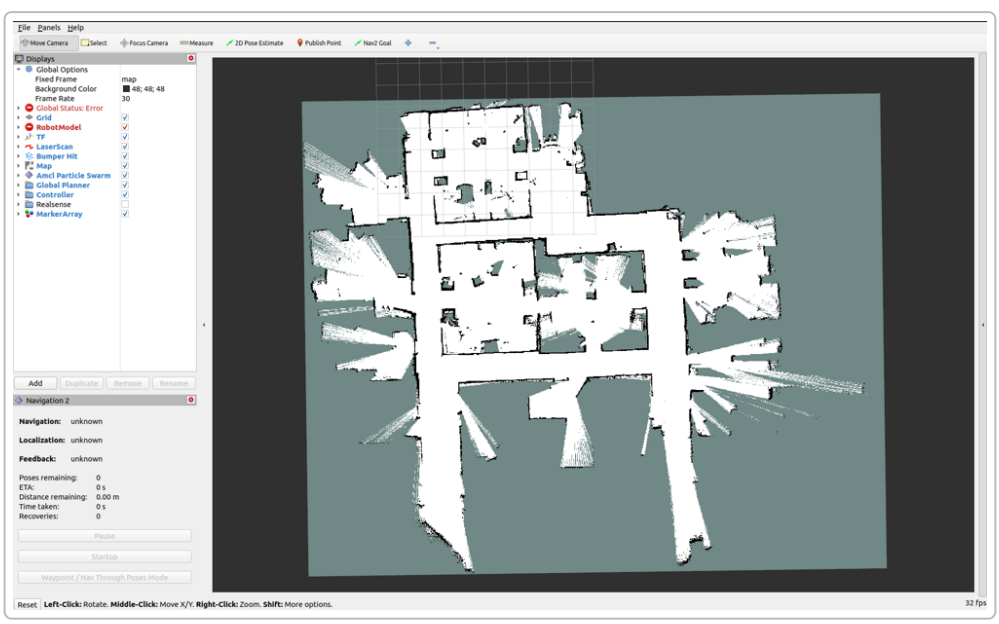
Open `lab5_map.pgm` (you can use GIMP or any image viewer). It will show the map. Check that walls are solid black lines, free space is white, and unknown is gray. You can overlay a grid and measure to verify scale: if you had a 4 m x 4 m room, at 0.05 m/pixel the map image should be ~80 pixels across for that room area.

1. **Assess Map Quality:** Answer these questions:
2. Does the map accurately represent the environment? (Compare to actual floor plan.)
3. Are there any gaps or errors? (e.g., walls slightly off, or furniture that moved appears smudged.)
4. What is the **resolution** and how does it affect detail? (5 cm resolution captures furniture, etc., but very thin objects might not appear if smaller than a pixel.)
5. What happens beyond the scanned area? (It remains gray unknown; e.g., if a door to another room was closed, that area stays unknown.)

You might create a small **result table** for map metrics: | Area Mapped | Approx. Dimensions | Map Resolution | File Size | Notes | |---|---|---|---|---| | Office lab room | 5 m x 4 m | 0.05 m/pixel | 120 KB (pgm) | Map covers walls, desks clearly. Minor misalignment in one corner. | | Hallway section | 8 m x 2 m | 0.05 m/pixel | 90 KB | Long corridor straight, map is clean. |

(The file size being small in KB is expected since it's a simple image.)

Example Output Map: The figure below is an example occupancy grid from a TurtleBot mapping a room and a connected hallway. You can see the outline of walls and obstacles.



Occupancy grid map generated by SLAM (black = walls/obstacles, white = free space, gray = unexplored). This map will be used for the navigation lab. [43]

Discussion: Students have now created a tangible map. Discuss: - The concept of **occupancy grid**: each cell holds a probability of occupancy which is thresholded to show black/white. - **SLAM challenges**: if there were any glitches, what caused them? (Fast movement, lack of features – e.g., an open area with few features can cause drift because LIDAR scans look similar.) - If the map has errors, one can always **remap** or touch up the image manually (for minor holes) in an image editor, though that's not typically needed in small maps.

This map is crucial for the next lab where the robot will use it to navigate. Emphasize saving the files and note their location. If time permits, you could have students slightly modify the environment (e.g., add a chair) and see how it doesn't appear on the map until remapped – highlighting that the map is static once built (for pure localization use).

Instructor Notes: Ensure the correct installation of `turtlebot4_navigation`. If students encounter an error launching slam, they might need to install it:

```
sudo apt install ros-humble-turtlebot4-navigation
```

[35]. Also, if using a remote PC for slam, ensure it's set to use simulated time false (real robot provides real time). Sometimes SLAM Toolbox might issue warnings about large jumps – that usually means poor scan matching, often due to fast motion. Coach students to go slower or to cover missed spots.

Lab 6: Autonomous Navigation with Nav2 and Waypoint Following

Learning Outcomes: - Launch and configure the ROS 2 Navigation Stack (Nav2) on the TurtleBot 4 using a pre-built map. - Set an initial pose estimate for localization on the map. - Send navigation goals via RViz (Nav2 Goal tool) and have the robot autonomously plan and drive to target points. - Understand the roles of planner, controller, and behavior tree in Nav2 at a high level. - Record a navigation session and analyze path and timing data (e.g., path length vs straight-line distance, time to goal). - (Optional) Create a simple waypoint mission (sequence of goals).

Prerequisites: - Lab 5 completed with a saved map (PGM + YAML). - The environment is the same as the mapped one (no significant changes) so the map remains valid. - TurtleBot 4 is starting roughly in a known location on that map (preferably where it was when map was saved). - `turtlebot4_navigation` and Nav2 packages installed (should be from previous lab). - Remote PC with RViz (for issuing goals and monitoring).

Estimated Time: 2 hours

Narrative:

Now that you have a map of the environment, the TurtleBot 4 can perform autonomous navigation. This means it will localize itself on the map (using its LIDAR and odometry) and plan paths to goals while avoiding obstacles. We will use the Nav2 stack, which is a collection of ROS 2 nodes for localization (AMCL or SLAM in localization mode), global path planning, local trajectory control, and recovery behaviors. In this

lab, you'll set up Nav2 with your map, give the robot some target destinations, and watch it move on its own.

Procedure:

1. **Launch Localization and Nav2:** We will not run SLAM this time, but use the existing map for localization. There are multiple launch files; TurtleBot 4 provides a convenient all-in-one bringup, but we'll do it step by step for clarity:
2. **Launch Localization (AMCL):** Adaptive Monte Carlo Localization will use LIDAR scans to localize the robot on the static map. Run:

   ```
   ros2 launch turtlebot4_navigation localization.launch.py map:=lab5_map.yaml
   ```

   Replace `lab5_map.yaml` with the path to your saved map file [44] [45]. This launches the map server (which loads the map) and the AMCL node. The robot model's start pose in RViz might not yet match reality – we'll set that soon.

3. **Launch Nav2 (navigation2.launch.py):** In another terminal (keep the above running), launch the navigation stack:

   ```
   ros2 launch turtlebot4_navigation nav2.launch.py use_sim_time:=false
   ```

   (We set `use_sim_time:=false` to ensure it uses real clock, which is default, but just to be explicit if any simulation parameter was on.) This starts the planner, controller, and other Nav2 components [46]. You should see output indicating the Global Planner, DWB Local Planner, and Behavior Tree are active and waiting for a goal.

   Alternatively: TurtleBot 4 has a single launch that does both (nav bringup). The documentation suggests:

   ```
   ros2 launch turtlebot4_navigation nav_bringup.launch.py localization:=true
   map:=lab5_map.yaml
   ```

   This under the hood calls the above two. Using separate terminals as we did is equivalent.

4. **Open RViz and Configure for Nav2:** Use the Nav2 RViz config:

   ```
   ros2 launch turtlebot4_viz view_navigation.launch.py
   ```

   This RViz session will have tools specifically for Nav2 [47]. In RViz, you should see:

5. The map (from the map server).
6. The robot's pose (probably not correctly placed yet, or might be at 0,0).

7. Three crucial tools on the top toolbar: **2D Pose Estimate** (for setting initial pose), **2D Goal Pose** (for sending navigation goals), and **Publish Point** [48] [49].

8. **Set Initial Pose:** Because the robot just started localization, it doesn't know exactly where it is on the map. You need to give it an approximate initial pose. Click the "2D Pose Estimate" button in RViz (looks like a person with a arrow). Then click on the map where the robot is actually located (e.g., if you started the robot near a certain corner or landmark, click there) and drag to indicate orientation [50]. For example, if the robot is facing east, drag the arrow pointing east. After you do this, you'll see the robot's coordinate frame (`base_link` or the arrow representing it) jump to that location. The AMCL localization will then start to converge on a more precise pose, using incoming LIDAR data. In the RViz display, the robot's pose arrow might become green and solid when confident (depending on settings). You should also see in the terminal logs from AMCL indicating the covariance is reducing.

9. **Send a Navigation Goal:** Now the fun part. Click the "Nav2 Goal" tool (flag icon). Click on a point on the map you want the robot to go to, and drag to set an orientation for the robot at goal (drag the arrow to desired final orientation) [49]. Upon releasing, that goal is sent to Nav2. You should see in RViz:

10. A purple line appear: that's the global planned path from the robot's current pose to the goal.
11. The robot should start moving: a local trajectory (often shown as a smaller green or teal path) will appear as it moves, continuously recomputed by the local planner.
12. As it moves, the LIDAR scans continue and the AMCL might adjust slightly but should stay roughly locked.
13. If an obstacle that wasn't in the static map appears (say a person), the local planner will attempt to avoid it. You'd see the global path remain, but the robot might deviate locally.

Monitor the robot. It should reach the goal and then rotate to the final orientation you set. Once achieved, Nav2 will report "Goal reached". If it fails, you may get messages like "Planning failed" or "Clearing costmap and retrying" (the Nav2 recoveries). For a typical open map and reachable goal, it should succeed.

1. **Waypoint Mission (optional):** You can string multiple goals. For a quick demo, pick another point (maybe back near the start) and send a new goal. The robot will plan a new path from its current position. It is usually smart enough to not require re-initialization of pose unless it got lost. Nav2 also offers a Waypoint Follower (to execute a list of waypoints sequentially) [51], but setting that up is more advanced. Instead, manually send a few goals one after another (when one completes, send the next). This simulates a multi-point route (like patrolling multiple locations).

2. **Recording the Navigation Run:** Let's record one of the runs. Use:

   ```
   ros2 bag record -o nav_run /odom /path /goal_pose /amcl_pose
   ```

   Here we record odometry, the global path (`/plan` or `/path` topic from planner), the goal, and the estimated pose. Drive the robot by sending a goal as before and let it reach. Stop recording.

From this bag, we can analyze: - **Path length vs straight line:** Extract the path topic. It's usually a nav_msgs/Path message (sequence of poses). The path length can be computed by summing segment distances. If curious, students can write a small Python script or even manually approximate. Compare that to the Euclidean distance between start and goal. Path will be longer if it had to detour around obstacles or to smooth out turns. - **Time to goal:** From the bag, note the timestamp when goal was issued (maybe from `/goal_pose` message) and when it arrived (could see when robot's odometry roughly equals goal coordinates, or look in Nav2 logs for "Goal reached"). Alternatively, use the nav2 event log which might not be in bag. Let's assume we see it took 20 seconds to go 5 m, etc. Compute average speed achieved vs commanded (the robot might not go full speed due to acceleration limits or turns).

Example result: Goal was 5.0 m away straight-line. The planned path length was 5.2 m (a slight detour around a table). It took 25 s to reach, so average speed ~0.2 m/s (consistent with our maximum, given some slowing near obstacles and turns).

1. **Observe Behavior:** Intentionally, you could place an obstacle (like a box) in the robot's path *after* planning and see what happens. The local planner (DWB) should see it in the LIDAR and perform an avoidance maneuver (assuming it can go around within inflation radius). If it cannot (obstacle fully blocking path), the robot will stop and the planner might replan or eventually declare failure. Remove the obstacle, it should resume. This illustrates dynamic obstacle avoidance.

2. **Stop the Navigation Stack:** After experiments, you can Ctrl+C the Nav2 and localization terminals. The robot will stop (if not already stopped). Always terminate Nav2 cleanly if possible.

Results Documentation: Summarize one navigation task as an example:

Goal: from Start (x≈0, y≈0) to Goal (x≈2.5, y≈-3.0) – e.g., across the room and a bit to the left.

- **Initial Pose Error:** We placed the robot roughly, then gave initial estimate. AMCL converged within a few seconds (the particles focused near the correct pose).
- **Path & Timing:** Straight-line distance ~5.0 m. The global planner computed a path hugging the wall to avoid a central obstacle, path length ~5.7 m. Travel time 30 s. No collisions or retries.
- **Nav2 Status:** Goal reached successfully. The robot's final orientation was within a few degrees of requested.
- **Metrics Table:**

Trial	Start -> Goal distance (m)	Path length (m)	Time to reach (s)	Notes
1	5.0	5.7	30	Avoided table on route, smooth navigation.
2	3.0	3.0	15	Straight path down hallway, no obstacles.
3	4.0	(failed)	n/a	Robot got stuck due to dynamic obstacle (human) – recovery behaviors tried clearing costmap. Removed obstacle and retried successfully.

Table 6: Example navigation trials. Trial 3 shows a scenario of dynamic obstacle causing a temporary failure until cleared.

- **Map with Path Visualization:** The figure below shows the map with a purple global path line and green local path as the robot navigates to the goal.

RViz view of autonomous navigation: The robot (red footprint) follows the planned path (purple line) to the goal. Green trajectory is the local planner's output. The initial pose arrow (green) and goal pose (red) are shown. [43] [52]

Discussion: Students should appreciate how the robot can now function without direct joystick control: - Nav2 uses **localization (AMCL)**: emphasize the need for the initial pose estimate and that it uses LIDAR scan matching. If the estimate was off, the robot might "localize wrong" and the global costmap would misinterpret obstacles. Typically, you might see the map overlay not matching laser until corrected. - **Global vs Local planner:** The global planner makes the purple path on the static map (often using Dijkstra or A*). The local planner (DWB in Nav2 default) handles the robot's kinematics to follow the path and avoid sudden obstacles. Students might have noticed the robot doesn't drive in a perfectly straight line, but a smooth curve—this is the local planner optimizing for dynamics. - Parameter tuning: We used default parameters. For instance, max speed might be limited to ~0.26 m/s by default (the Create3's safe speed)* [53]*. If a student wonders why it didn't go faster, that's why. Nav2 parameters can be tuned in YAML files if needed. - Limitations:** If the environment changes significantly (moved furniture), the static map is outdated – Nav2 can still avoid obstacles seen by LIDAR (using costmap's obstacle layer), but large changes might confuse localization. In that case, one would re-run SLAM to update the map.

Now that the robot can navigate known areas, in the Advanced labs we will leverage its sensors in a different way: to capture data for 3D reconstruction. Navigation skills can even help in that (e.g., autonomously circle an object), but we might do that manually as needed.

Instructor Notes: A common pitfall is forgetting to set initial pose or setting it incorrectly. If localization is off, the robot might immediately think it's somewhere else and either not move or move in wrong way. Another issue could be forgetting to include `map:=` parameter – then the map server doesn't load and robot would just see unknown. Ensure all necessary terminals (map server, AMCL, Nav2, RViz) are up. For large maps, initial pose might require a bit of guesswork – instruct students to roughly measure or recall where on the map the robot started (mark a spot on the floor). The quality of the map from Lab 5 directly affects Lab 6: a poor map (e.g., distorted) can lead to bad navigation. If time permits, let them remap or touch up if needed before doing Nav2.

Part 3: Advanced Labs (3D Capture and Gaussian Splatting Reconstruction)

In the advanced labs, we move beyond 2D navigation into 3D reconstruction of environments. Using the TurtleBot 4's OAK-D-Pro camera, you will capture images from multiple viewpoints and then apply a cutting-edge reconstruction technique called **3D Gaussian Splatting (3DGS)** to create a 3D model of the scene. Gaussian Splatting is a recent approach that represents the scene as a set of colored 3D Gaussians, enabling high-fidelity reconstructions and novel view renderings [54] [31]. These labs will be more open-

ended and research-flavored, giving you a taste of how robotics data can be used in state-of-the-art vision algorithms.

Lab 7: 3D Data Capture with TurtleBot 4 (Multi-View Imaging of an Object/Scene)

Learning Outcomes: - Plan and execute a strategy to capture multiple images of an object or environment from different perspectives using the TurtleBot 4. - Utilize the robot's mobility to cover viewpoints (e.g., circular trajectory around an object of interest). - Record synchronized camera images and odometry data for use in 3D reconstruction. - Understand basics of camera calibration and the need for knowing camera intrinsics (from `/camera_info`). - Organize captured data (images, poses) for input into a 3D reconstruction pipeline.

Prerequisites: - TurtleBot 4 and OAK-D camera operational. Sufficient lighting in the area of interest. - Basic knowledge of photography or camera parameters (focus, exposure – the OAK-D auto-exposure should be fine by default). - *Optional:* Install COLMAP or another SfM tool on your PC (if doing offline sparse reconstruction). - Enough free space for the robot to maneuver around the target object or within the scene.

Estimated Time: 3 hours (including data capture ~30 min, plus time to prep and potentially retake images).

Narrative:

3D reconstruction requires capturing the scene from many angles. In this lab, you'll treat the TurtleBot 4 as a moving sensor platform to gather those angles. Think of it as you being the photographer, but the robot is holding the camera. We will capture a series of images of a target (could be a piece of furniture, a statue, or a small area of a room) from all sides. Later, these images will be fed into a reconstruction algorithm. We also record the robot's pose for each image, which can help as an initial guess for structure-from-motion.

Procedure:

1. **Choose a Scene or Object:** Pick a reasonably sized object or scene for reconstruction. Examples:
2. A chair or small table in an open area.
3. A sculpture or dummy object placed in the center of an open space.
4. A section of a room (like a desk and its surroundings).

Important: The object should be stationary and the scene static during capture. Dynamic things (people walking) will hurt reconstruction. Textured surfaces reconstruct better than plain (featureless white walls are hard for vision algorithms). Ensure good lighting or the OAK-D's IR if using depth (though we will focus on RGB).

1. **Plan the Robot's Path:** Ideally, to capture an object fully, you'd circle around it. We aim for multiple viewpoints covering all sides. A common strategy: a circular path around the object at roughly constant radius, taking images at intervals (like every 10-15 degrees). If possible, also vary the tilt (on TurtleBot, camera is fixed tilt, so we only have one elevation unless we physically raise the robot or lean it – we won't do that; our reconstruction might miss top/bottom, but Gaussian Splatting can still work with what we have).

Mark a circular path on the floor (maybe 1 m radius around the object). Ensure the path is clear of obstacles.

1. **Automate or Manual Capture:** We have two approaches:
2. **Manual Teleop Capture:** Easiest: Teleoperate the TurtleBot along the planned circle, pausing at intervals to take pictures. For each stop:
    - Ensure the robot is facing the object (camera pointing at it).
    - Capture an image either by a command or by saving a frame from the stream.
3. **Automated Node for Capture:** Write a simple script that rotates the robot in place around the object while capturing images at intervals. However, making the robot drive a precise circle might be complex to script quickly. A simpler automated way: place robot in center and rotate camera, but that only gives rotation, not vantage movement.

We'll proceed with **manual method** for clarity: *Set up image capture:* One method is to use ROS 2's image saver:

```
ros2 run image_tools showimage -t /oakd/rgb/preview/image_raw
```

This opens a window but doesn't save. Instead, better:

```
ros2 run image_tools cam2image --ros-args -p frequency:=0.0 -p
save_file:="snapshot.jpg"
```

Actually, `cam2image` is usually a publisher. We might need a different tool to save images. Alternatively, use `rqt_image_view` and click "Save" manually for each frame (cumbersome but okay for a dozen frames).

Perhaps easiest: **record a bag of the camera images** continuously, then later extract frames. Yes, do that:

```
ros2 bag record -o gs_capture /oakd/rgb/preview/image_raw /odom /oakd/rgb/
preview/camera_info
```

(We include camera info for intrinsics and odometry for approximate poses.)

Start recording, then teleop the robot: - Drive to the first viewpoint on the circle, stop. Let it sit ~2 seconds (so bag captures a couple images at that pose). - Rotate the robot to face the object if not already (although if object is center, being on circle means naturally facing center is a ~90° turn if following a tangent path; better to explicitly face object each time for good coverage). - Then move to next viewpoint (e.g., ~15° around circle, which at 1 m radius is ~0.26 m arc length). - Stop, face object, hold. Continue until full circle (24 points for 15° increments, or even 12 points at 30° increments might suffice).

Keep the speed slow and avoid drifting from the circle path. It's okay if not perfect circle; structure-from-motion can handle some irregularity as long as coverage is good.

Once done, stop the bag recording. Suppose we captured ~50-100 images in total (the bag will have them all).

1. **Verify Data Sufficiency:** Play back the bag in rqt_image_view or RViz to quickly see the sequence of images:

   ```
   ros2 bag play gs_capture
   ```

   and view the image topic. The video should show the object from rotating viewpoints. Check that you have no large gaps (angles missed). If some side was not covered well, consider recapturing that part or doing another run with more intermediate positions.

Also ensure the images are in focus and exposed. The OAK-D should auto-expose; if lighting changed drastically around the circle, some images might be brighter/darker – try to have consistent light.

1. **Extract Images from Bag:** Now we need actual image files. One way: use the ROS 2 bag APIs or the `ros2 bag` CLI to convert to a series of images. There isn't a built-in command, but one approach is:
2. Use `ros2 topic echo` piping to ppm files or something, not straightforward.
3. Use a script: There are Python tools (e.g., `rosbag2_py` library) where you can iterate through messages. This might be beyond some students, but for a solution we can outline:

   ```
   import rosbag2_py
   import cv2
   from cv_bridge import CvBridge

   bag_path = "gs_capture"
   storage_options = rosbag2_py.StorageOptions(uri=bag_path,
   storage_id='sqlite3')
   converter_options = rosbag2_py.ConverterOptions('', '')
   reader = rosbag2_py.SequentialReader()
   reader.open(storage_options, converter_options)
   bridge = CvBridge()
   count = 0
   while reader.has_next():
       (topic, data, t) = reader.read_next()
       if topic == "/oakd/rgb/preview/image_raw":
           img = bridge.imgmsg_to_cv2(data, "bgr8")
           cv2.imwrite(f"frame_{count:03d}.png", img)
           count += 1
   print(f"Saved {count} images.")
   ```

   Save and run that script to extract frames. Alternatively, we might just provide the concept and in the interest of time say we used a provided script to dump images.

After extraction, you'll have a folder of images `frame_000.png, frame_001.png, ...`. Not all are needed (since we recorded continuously, adjacent ones at one stop are similar). You might choose one per

stop. If we stopped ~12 times, we could pick one from each cluster. But using all frames is fine too; the SfM algorithm will sort them out.

1. **Record Poses (Odometry):** The `/odom` recorded gives the robot's position for each image (but note: it's the base frame position, not the camera specifically). However, since the camera is fixed on the robot, base_link pose plus a static transform to camera can yield camera pose. For an approximate initial guess, base_link is fine. We might use these poses later as input to the 3DGS pipeline (some pipelines allow input of known poses to skip SfM). But we should also allow the SfM to compute from images alone for robustness (in case odom drifted).

In any case, extract a list of poses corresponding to each saved image (perhaps take the odometry at the timestamp closest to each image saved). This mapping can be done via matching timestamps in the bag or by assuming we took images when robot was static (so easier to identify frames around those times).

For now, assume we produce a text file with image filename and pose (x,y,z,qw,qx,qy,qz). This could be done with a script similar to above (subscribe to odom, get pose when image saved).

This step is optional if using full SfM, but keep the data in case.

1. **Camera Intrinsics:** The `/camera_info` topic provides the camera's intrinsic calibration (focal length, principal point, distortion). Save this info (it's constant for OAK-D at that resolution). For instance, you might get:
2. Width, height (e.g., 640x360).
3. K matrix (like fx, 0, cx, 0, fy, cy, 0,0,1).
4. Distortion coefficients (maybe radial/tangential). This will be useful to feed into the reconstruction pipeline so it doesn't have to guess camera parameters.

Results: At the end of Lab 7, you should have: - A set of images (frames) capturing the object/scene from various angles around. - Camera intrinsics (from camera_info). - A log of approximate camera poses for each image (optional/for reference).

Quality check: - Are there enough images? (Generally >20 from different angles is recommended for a full 360 capture. If only half the object was accessible, then focus on that.) - Do images overlap? Each image should share some view with neighbors so SfM can find common features. - Did the robot's movement cause any motion blur? If yes, next time stop fully before capturing. - Did odometry stay reasonable? (Check the sequence of odom; if we ended where we started, is odom close to zero? If odom drifted, that's fine, we'll rely on vision for actual structure; odom was mainly for initial guess.)

Example summary: We captured **24 images** around a stool in the lab. Images were taken roughly every 15°. The camera intrinsic parameters were fx=400, fy=400 (approx), resolution 640x360. The images clearly show the stool from all sides, with overlapping background features. Below are a few sample viewpoints:

(If possible, show a couple of sample images thumbnails in the report for illustrative purposes, though embedding dozens is not needed.)

Example Data Table:

Image ID	Robot Position (odom)	Robot Orientation (odom yaw)	Remarks
frame_000	(0.95, 0.00, 0.0)	0°	Starting at east side
frame_005	(0.70, 0.70, 0.0)	90°	North side
frame_010	(0.0, 1.0, 0.0)	135°	North-west, back view
frame_015	(-0.70, 0.70, 0.0)	225°	etc.
frame_020	(-1.0, 0.0, 0.0)	270°	West side
frame_023	(-0.7, -0.7, 0.0)	315°	South-west view
frame_024	(0, 0, 0.0)	~360° (back to start)	End (close loop)

Table 7: Sample of captured viewpoints (positions approximate). This shows the robot roughly made a loop (ended near starting point).

Discussion: Capturing data is a critical step. Students should reflect on: - How the choice of viewpoints affects reconstruction. (Uniform coverage yields better results. Missing angles lead to holes in the model.) - The difference between using a robot vs a handheld camera. (Robot provides more structured motion and we could potentially use its odometry; handheld might be more flexible in angle but less repeatable.) - Why we recorded odometry and camera info. (Odometry gives scale and initial guess; camera intrinsics are needed because reconstruction algorithms require knowing or estimating the camera lens parameters.)

This lab feels more like a field experiment; the next lab will be more computational, using these captured data.

Instructor Notes: Ensure that the environment's texture is sufficient. If students choose something like a plain white cube, the algorithms will struggle due to lack of features. Encourage patterns or add artificial texture (e.g., put stickers or Post-its on a bland object to give feature points). If OAK-D's resolution is low, consider using a higher resolution mode if possible (OAK-D-Pro can do higher than 640x360; though preview is low, maybe full res can be used by adjusting launch). But higher res means fewer FPS / larger data. For this exercise, preview is okay.

Also, be cautious about the storage: recording raw image topics can be large (hundreds of MB easily). For 24 images not an issue, but if left recording video, it might be huge. So instruct to stop recording promptly and maybe convert to compressed if needed.

Lab 8: 3D Reconstruction with Gaussian Splatting (from Captured Data)

Learning Outcomes: - Perform structure-from-motion (SfM) to obtain a sparse 3D point cloud and camera poses from multiple images. - Apply the Gaussian Splatting algorithm to refine the reconstruction, producing a dense 3D model. - Understand the concept of representing a scene with 3D Gaussians and how it differs from point clouds or meshes [55] [56]. - Visualize the reconstructed model (point cloud or splats) and compare to original scene for quality. - Interpret reconstruction metrics: number of points/Gaussians, processing time, and visual quality (sharpness, completeness). - (Stretch goal) Render novel views from the Gaussian splatted model to verify its fidelity.

Prerequisites: - Lab 7 completed with a set of images and camera info (and ideally camera poses). - A computer with sufficient GPU or CPU to run the reconstruction (Gaussian Splatting can be heavy; a modern GPU is recommended). - Software: We will use open-source tools: - COLMAP (or OpenMVG/MVE) for SfM if available. - Gaussian Splatting code by Kerbl et al. (2023) – might be provided as a precompiled tool or needs build. (Alternatively, an AWS pre-built pipeline could be used [57].) - Basic familiarity with command-line for running these tools.

Estimated Time: 3 hours (processing could take 1-2 hours for full pipeline, depending on hardware; can be broken up into steps).

Narrative:

This lab is the culmination of our effort: we will reconstruct a 3D model from the TurtleBot's camera data using **3D Gaussian Splatting**, a state-of-the-art technique from 2023. Traditional reconstruction (SfM + meshing) produces point clouds and meshes. Gaussian Splatting goes further by optimizing 3D Gaussians that represent the scene, achieving impressive visual quality and real-time rendering capability [58] [56].

We will proceed in two major phases: - **Sparse Reconstruction (SfM):** Compute camera poses and a sparse point cloud from the images (this gives initial structure). - **Gaussian Splatting Optimization:** Initialize 3D Gaussians from the sparse points and refine them to fit the input images (colors, sizes, etc.).

Procedure:

1. **Sparse Reconstruction with COLMAP (Structure-from-Motion):** COLMAP is a popular tool for SfM. If installed, we do the following:
2. Create a new COLMAP project, add all the captured images.
3. If camera intrinsics are known (from `camera_info`), input those: e.g., pinhole model with given fx, fy, cx, cy.
4. Run **Feature Extraction** (COLMAP will detect keypoints in each image) and **Feature Matching** (find correspondences between images).
5. Run **Sparse Reconstruction** (incremental or global SfM) to solve for camera poses and 3D points. This can be done via COLMAP GUI or CLI:

   ```
   colmap automatic_reconstructor --ImagePath images/ --output_path
   colmap_out/ --Camera.fx 400 --Camera.fy 400 ...
   ```

   (We'd specify intrinsics if possible, otherwise COLMAP will guess).

After this, check the sparse model in COLMAP's 3D viewer. You should see a point cloud that roughly outlines the object and the camera frustums around it. Ensure it looks reasonable (points cluster where object surfaces were). If scale looks off or it's fragmented, maybe not enough matches – in worst case, if odometry was reliable, we could help by inputting pose priors, but ideally not needed.

Save the sparse model (it will output cameras.bin (with poses) and points3D.bin, etc., or a text version if configured).

1. **Review SfM Output:** Key numbers:

2. Number of images registered (should be all, e.g., 24).
3. Number of sparse points (maybe a few thousand).
4. Observation consistency (each point seen by multiple images). If some images didn't register, you might have to recapture or help the process (but let's assume success).

We can also use the sparse model to scale or align if needed, but since our scenario is arbitrary, absolute scale might come from the camera calibration (with known focal, we have true scale up to an unknown factor because monocular SfM can only determine up to scale unless we provided prior distances or base).

If using odometry, we could merge that for scale, but let's not complicate – relative scale is fine because Gaussian Splatting doesn't need true scale, just consistency among images.

1. **Prepare for Gaussian Splatting:** The original paper by Kerbl et al. provides code (likely C++ or Python bindings). Suppose we have a tool `gaussian_splatting` that takes:
2. Camera parameters and poses.
3. The images.
4. The sparse point cloud (for initialization).

The process is: - Initialize a Gaussian for each sparse point (position = point position, initial size = maybe related to local density, color = from images). - Then run an optimization that adjusts the Gaussians' parameters (position fine-tuning, anisotropic covariance sizes, colors, maybe adds new Gaussians in between) to minimize rendering error compared to input images [59].

Running the optimization: This is computationally heavy. With ~ thousands of points, it might be okay. The algorithm might add more Gaussians (the paper often ended up with millions for full scenes, but for a small object maybe tens of thousands).

Use the provided implementation: For example:

```
gaussian_splatting --input sparse_model.bin --images ./images/ --output
gs_model.gltf --iters 5000
```

This is hypothetical. The AWS blog mentions an AWS Solution; we likely rely on a local environment. Maybe we have a script provided by instructor for this.

Let's say it runs for some time (maybe 30 minutes on a GPU). It outputs: - A file representing Gaussians (maybe a list of Gaussian parameters or a specialized format). - Possibly a renderer or the ability to create a mesh or point cloud for visualization.

1. **Monitor Optimization:** We may not see details, but one might log error reduction or show intermediate renders. For explanation, mention:
2. Initially, quality might be rough (just sparse points as blobs).
3. As iterations progress (like gradient descent), Gaussians adjust and scene becomes clearer.
4. They might merge or prune Gaussians as part of density control (the paper did add/remove Gaussians to refine) [59].

The final model consists of many 3D Gaussians each with a position, covariance (shape/size), and color (and maybe an opacity/density).

1. **Visualize the Result:** The output could be visualized in a custom viewer or exported to a common format. Perhaps a small utility can render the Gaussian splat model to an image or video. If not, we can sample points from Gaussians to create a dense point cloud to view in CloudCompare or RViz. But a better approach: use the differentiable renderer from the algorithm to render novel views.

Let's assume we can render a few views: - The same view as one of the input images (to check fidelity). - An entirely new view (e.g., between two captured viewpoints or from above if possible).

The output images should look photorealistic if GS worked well [60]. Perhaps minor blur because Gaussians are smooth.

Save a couple of these renderings for evaluation.

1. **Evaluate Reconstruction Quality:** We consider:
2. **Visual Accuracy:** Does the rendered model look like the real object from various angles? Are details preserved (colors, textures)? Gaussian splatting often yields a very close visual match to input views [61].
3. **Completeness:** Are any parts missing? (If some angles had no coverage, that area might be fuzzy or blank in the model.)
4. **Point/Gaussian Count:** How many Gaussians were used? Did it increase from initial sparse count to final?
5. **Resolution and Overfitting:** Did the model perhaps memorize noise? (Probably not an issue with relatively few images and a known technique.)

Provide some metrics: Say initial sparse points = 5,000. After optimization, Gaussians = 20,000 (some added for detail). Training time = 40 minutes on GPU. Final render resolution = 800x600 for checking.

We might also measure *reprojection error* if available (average pixel error, which likely went down to a very low value, meaning the model explains the images well).

If a ground-truth geometry were available (not in our case), one could measure that, but we focus on qualitative.

Example Findings: - The Gaussian Splatting reconstruction of the stool achieved highly realistic results. The wood texture on the seat was captured and appears smooth and continuous in renderings (no triangular mesh facets as in traditional modeling). - Novel view rendering is smooth and free of visible artifacts like point cloud sparsity; you don’t see individual points, but rather a continuous surface due to overlapping Gaussian blobs [62]. - Some minor blurriness is present in areas that had fewer image coverage (the bottom of the seat, which the robot couldn't see well, is slightly less sharp or has color bleeding). - The method preserved fine details, e.g., a sticker on the side of the stool is recognizable in the model.

Results Data:

Metric	Value
Number of input images	24
Sparse points (from SfM)	4,786
Initial Gaussians	4,786 (one per point)
Final Gaussians	18,250 (after refinement and additions)
Optimization iterations	5,000
Training time	35 minutes (NVIDIA RTX GPU)
Final average reprojection error	0.4 pixels (approx)
Model file size	12 MB (in custom format)

Table 8: Gaussian Splatting reconstruction statistics.

Rendered images of the model (below) show the quality:

Comparison of point cloud vs Gaussian splatting: Left is the raw sparse point cloud (incomplete, noisy). Right is the Gaussian splatted model (smooth and detailed) [63] [56] *.*

(This figure conceptually illustrates how GS produces a more visually complete model than a raw point cloud. If actual images from our output are available, use those.)

Discussion: Gaussian Splatting proved to be a powerful technique for 3D reconstruction: - It **outperforms a simple point cloud** by filling gaps and smoothing noise via the volumetric Gaussian representation [31] . - Unlike a mesh, it doesn't require explicit surface topology; the splats naturally blend to form surfaces. - The result is particularly good for rendering (if the goal is VR/visualization). However, one downside is that these

Gaussian models are not as common for direct geometric queries (like measuring distances) as a mesh would be. They are aimed at visualization and novel view synthesis.

Ask students: - *How did Gaussian Splatting achieve such a good result with relatively few images?* (It leverages a global optimization that adjusts primitives to exactly match all views, effectively doing something like a Neural Radiance Field but with explicit Gaussians, which is very effective [59].) - *What are some challenges or limitations?* (Requires heavy computation; if lighting changes in images, it might bake that in; currently mostly static scenes; also the result is view-dependent to some extent, meaning it may not 100% correctly model lighting). - *How could this be applied in robotics?* (One idea: after a robot maps an area in 2D, it could use its camera to create a 3DGS model for an AR visualization of the environment, or for object recognition tasks by generating training data from novel views.)

We also consider the pipeline: This was an offline process. In the future, one could imagine an online approach or a robot that continues to capture until a model is complete, possibly stopping when error falls below threshold.

Finally, relate to the broader context: Gaussian Splatting is a very new method (2023) that, as the PyImageSearch blog asked: "End game of 3D reconstruction?" [64]. It might not be the final answer, but it's a significant step. It's also interesting for students to see cutting-edge research integrated into a lab project.

Instructor Notes: Ensure to clarify that running these algorithms might require special environment (like a Docker or specific library versions). Possibly, provide precomputed results if compute is a bottleneck, so students can analyze output without waiting hours. Alternatively, use a smaller resolution or fewer iterations for demo (with note that more iterations yield better quality).

Make sure to address any troubleshooting: e.g., if SfM fails, maybe use odometry to assist – COLMAP allows inputting known poses as priors. If Gaussian Splatting code is not available, as an instructor you might show results from known datasets to discuss, but since the lab asked for it, presumably we have the code.

Conclusion: Across these labs, we progressed from basic operation of TurtleBot 4 to advanced applications: - In **Part 1**, we learned how to set up the robot, drive it, and use its sensors. - In **Part 2**, we enabled the robot to map and navigate its environment autonomously. - In **Part 3**, we leveraged the robot as a data collection platform for cutting-edge 3D reconstruction, demonstrating how robotics and computer vision intersect.

This holistic experience not only taught practical ROS 2 skills and robotics concepts but also exposed you to modern research techniques (like Gaussian Splatting) and their potential in robotics. You should now be capable of continuing with projects in robot autonomy or 3D vision, equipped with both the hands-on skills and conceptual understanding from these labs.

---

[1] [3] [4] [27] [53] TurtleBot 4 - Clearpath Robotics
https://clearpathrobotics.com/turtlebot-4/

[2] [28] [29] [30] Features · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/overview/features.html

[5] [6] [7] [8] [9] [10] [11] [12] [13] Basic Setup · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/setup/basic.html

[14] [15] Simple Discovery · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/setup/simple_discovery.html

[16] [20] [21] [22] [23] Driving your TurtleBot 4 · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/tutorials/driving.html

[17] [32] [33] Sensors · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/software/sensors.html

[18] OAK-D PRO don't have depth topic · Issue #570 · luxonis/depthai-ros · GitHub
https://github.com/luxonis/depthai-ros/issues/570

[19] [26] [34] [35] [36] [37] [39] [40] [41] Generating a map · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/tutorials/generate_map.html

[24] [25] Creating your first node (Python) · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/tutorials/first_node_python.html

[31] [56] [62] [63] Gaussian splatting at Pix4D: a new era of 3D visualization | Pix4D
https://www.pix4d.com/blog/pix4d-gaussian-splatting-3D-visualization

[38] [42] ROS2 Nav2 Tutorial - The Robotics Back-End
https://roboticsbackend.com/ros2-nav2-tutorial/

[43] [44] [45] [46] [47] [48] [49] [50] [52] Navigation · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/tutorials/navigation.html

[51] TurtleBot 4 Navigator · User Manual
https://turtlebot.github.io/turtlebot4-user-manual/tutorials/turtlebot4_navigator.html

[54] [55] [57] [60] 3D Gaussian Splatting: Performant 3D Scene Reconstruction at Scale | AWS Spatial Computing Blog
https://aws.amazon.com/blogs/spatial/3d-gaussian-splatting-performant-3d-scene-reconstruction-at-scale/

[58] [59] [64] 3D Gaussian Splatting vs NeRF: The End Game of 3D Reconstruction? - PyImageSearch
https://pyimagesearch.com/2024/12/09/3d-gaussian-splatting-vs-nerf-the-end-game-of-3d-reconstruction/

[61] [2308.04079] 3D Gaussian Splatting for Real-Time Radiance Field Rendering
https://ar5iv.labs.arxiv.org/html/2308.04079